

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

- Uma **articulação** de um grafo conexo G é um vértice v cuja remoção desconecta G .
- Se v é articulação então $\omega(G-v) > \omega(G)$, isto é, o número de componentes conexas de $G-v$ é maior do que o número de componentes conexas de G .

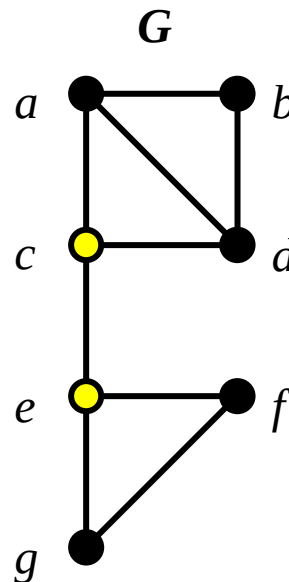
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

- Uma **bloco** de um grafo G é um subgrafo maximal H de G com a seguinte propriedade: H (considerado isoladamente) é conexo e não contém articulações.
- Em alguns casos, um bloco pode ser formado por uma única aresta. Esta aresta será chamada **ponte**.
- Em todo bloco *que não seja uma ponte*, existem dois caminhos internamente disjuntos entre qualquer par de vértices. Neste caso, o bloco é um **subgrafo maximal biconexo**.

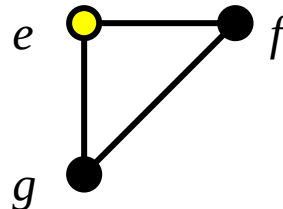
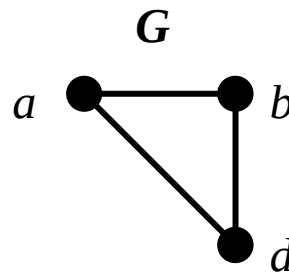
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



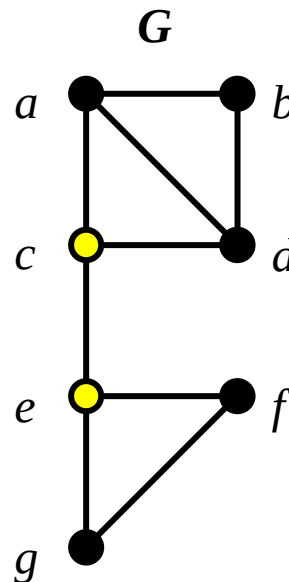
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



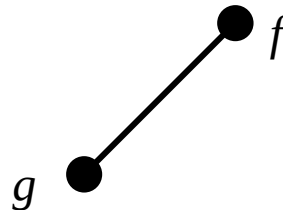
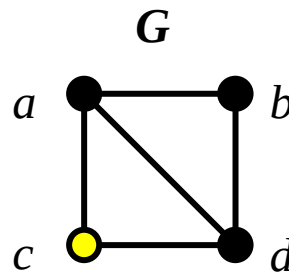
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



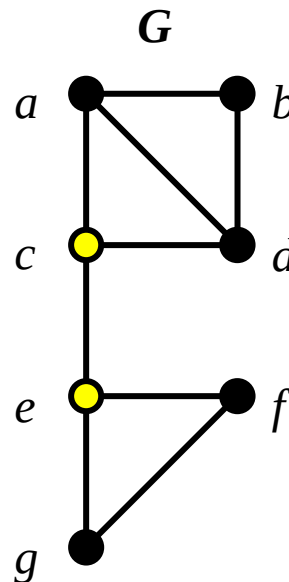
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



Busca em profundidade

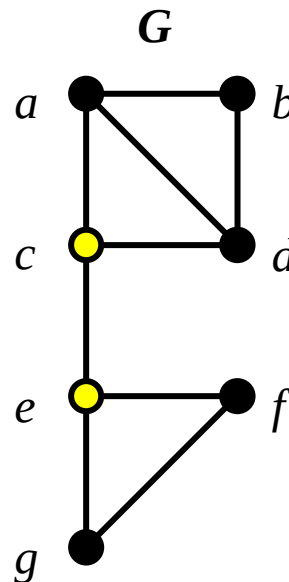
Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



Busca em profundidade

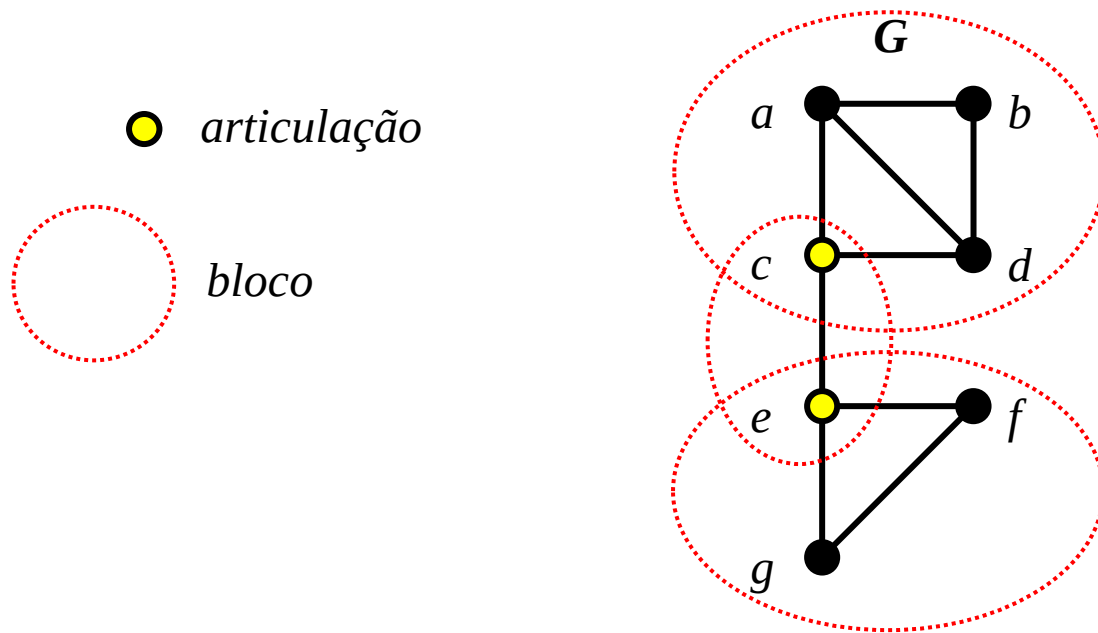
Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

● *articulação*



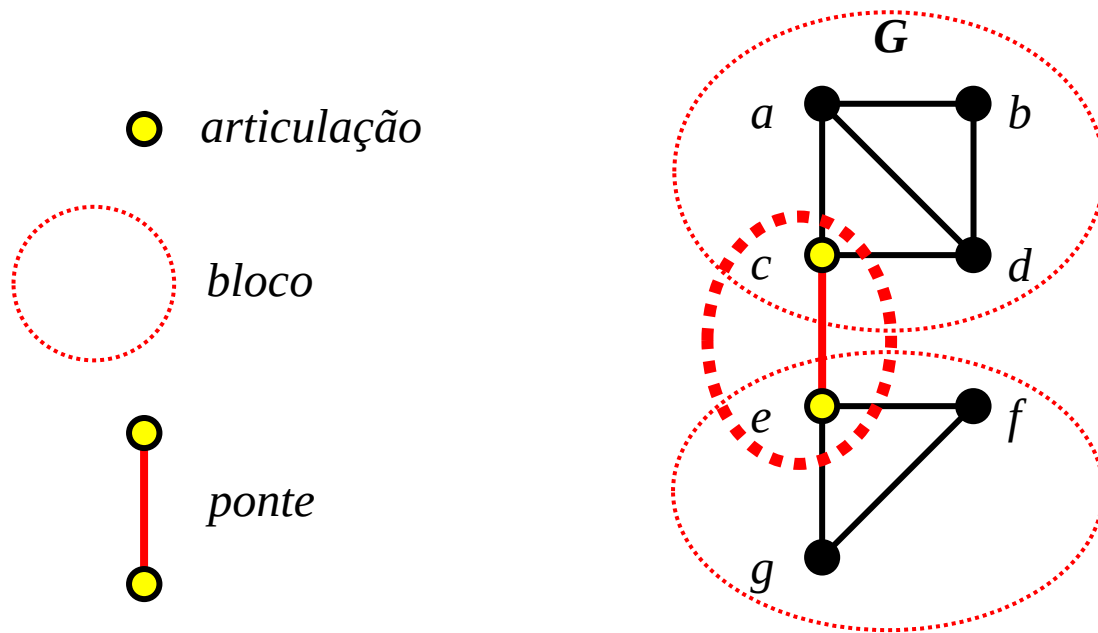
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



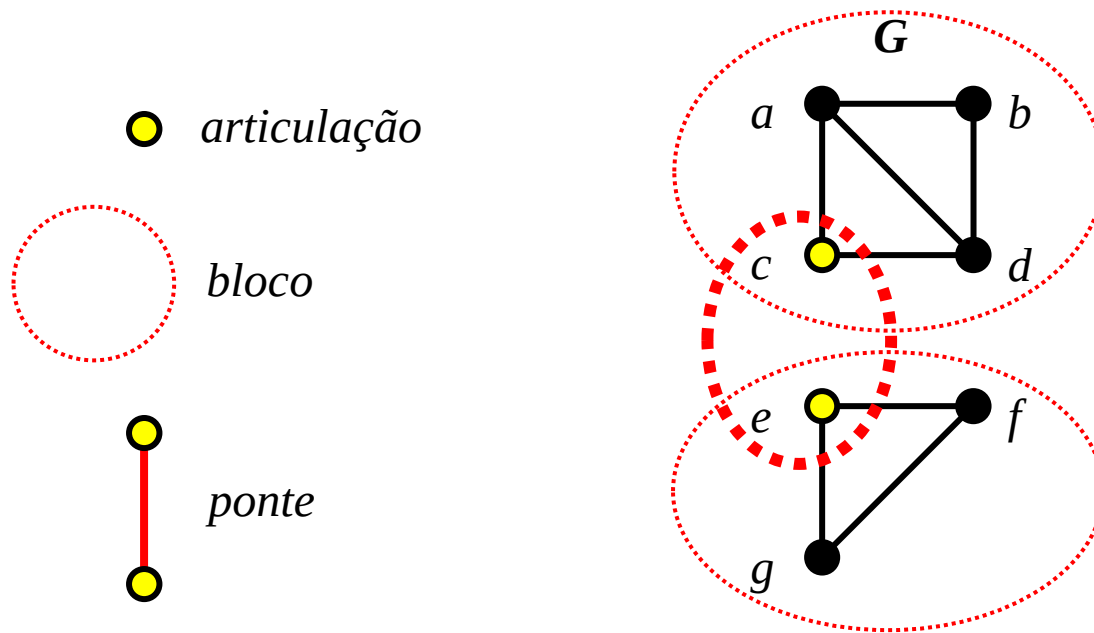
Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .



A remoção de uma ponte desconecta o grafo!

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

- Os blocos de um grafo G determinam naturalmente uma *partição do conjunto de arestas* de G , isto é, cada aresta pertence a um e apenas um bloco de G .
- O mesmo não ocorre em relação aos vértices:
 - se v pertence a mais de um bloco então v é uma articulação
 - se v pertence a um único bloco então v **não** é uma articulação

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

- Considerando o grafo como uma rede ...
 - articulações são **nós críticos**
 - pontes são **conexões críticas**

Busca em profundidade

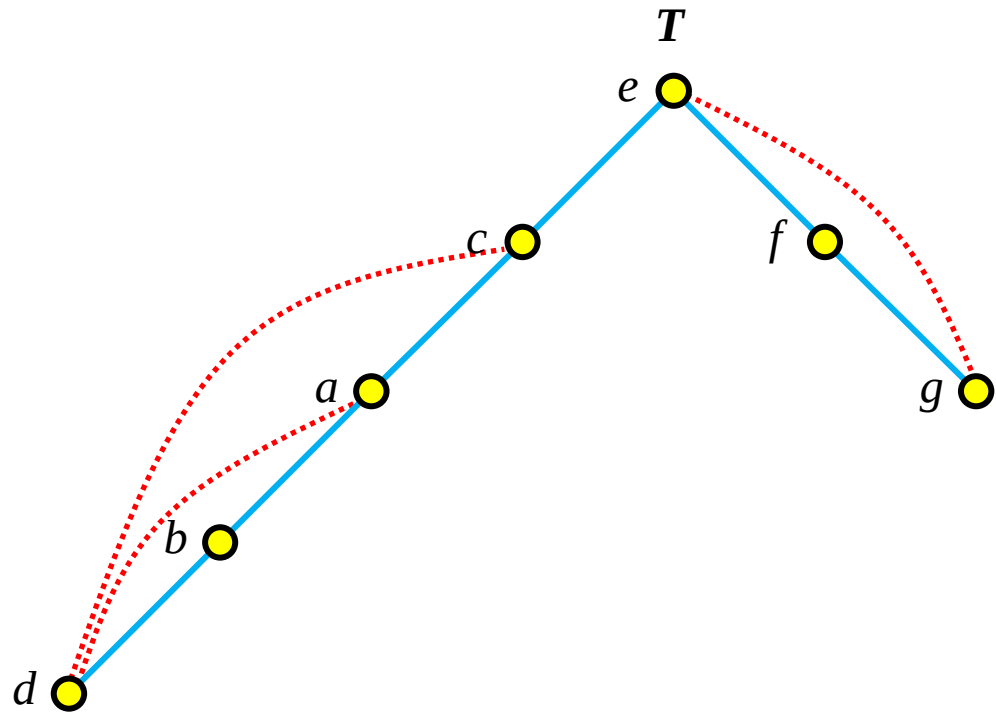
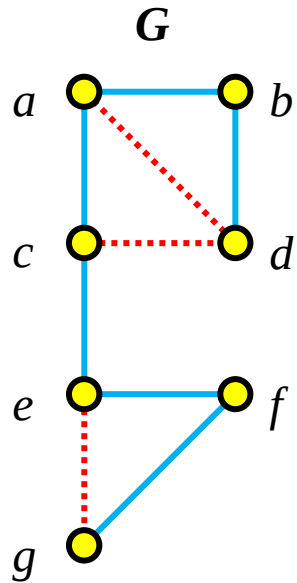
Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

Definição:

Seja T uma árvore de profundidade para o grafo G .

$back(v) = PE$ do vértice w mais próximo da raiz de T que pode ser alcançado a partir de v usando **0 ou mais** arestas de T para baixo e, a seguir, **no máximo uma** aresta de retorno.

Busca em profundidade



v	a	b	c	d	e	f	g
$PE(v)$	3	4	2	5	1	10	11
$PS(v)$	8	7	9	6	14	13	12
$back(v)$	2	2	2	2	1	1	1

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

Como calcular $back(v)$

$$back(v) = \min(\{ PE(v) \} \cup \\ \{ back(w) \mid w \text{ é filho de } v \text{ em } T \} \cup \\ \{ PE(w) \mid vw \text{ é aresta de retorno} \})$$

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

procedimento $P(v)$ -- *com cálculo de $back(v)$*

$t \leftarrow t + 1$; $PE(v) \leftarrow t$; **$back(v) \leftarrow PE(v)$** ; *=> inicialização*

para todo vértice w em $N(v)$ faça

se $PE(w) = 0$

então | visitar vw ; $pai(w) \leftarrow v$;

| executar $P(w)$

| **$back(v) \leftarrow \min(back(v), back(w))$**

senão se $PS(w) = 0$ e $w \neq pai(v)$

então | visitar vw

| **$back(v) \leftarrow \min(back(v), PE(w))$**

fim-para

$t \leftarrow t + 1$; $PS(v) \leftarrow t$

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

Como usar os valores $back(v)$ para determinar as articulações

Teorema: Seja T uma árvore de profundidade para um grafo G . Suponha que os valores $PE(v)$ e $back(v)$ estejam calculados. Seja v um vértice qualquer de G .

- Se v é a raiz de T então v é articulação sss v possui dois ou mais filhos em T .
- Se v **não** é a raiz de T então v é articulação sss existe pelo menos um filho w de v com $back(w) \geq PE(v)$.

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

procedimento $P(v)$ -- *com cálculo de $back(v)$ e dos blocos (idéia: usar uma pilha)*

$t \leftarrow t + 1$; $PE(v) \leftarrow t$; **$back(v) \leftarrow PE(v)$** ; $\Rightarrow$ *inicialização*

para todo vértice w em $N(v)$ faça

se $PE(w) = 0$

então | visitar w ; **$empilhar\ w$** ; $pai(w) \leftarrow v$

| executar $P(w)$

| **se $back(w) \geq PE(v)$ então desempilhar e imprimir tudo até w**

| **$back(v) \leftarrow \min(back(v), back(w))$**

senão se $PS(w) = 0$ e $w \neq pai(v)$

então | visitar w ; **$empilhar\ w$**

| **$back(v) \leftarrow \min(back(v), PE(w))$**

fim-para

$t \leftarrow t + 1$; $PS(v) \leftarrow t$

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

Algumas questões:

- Se a busca em profundidade se iniciar por um vértice que *não é* articulação, então a raiz da árvore de profundidade terá apenas um filho! (veja o Teorema)
- **Exercício:** Simular a execução para observar o comportamento da pilha de arestas.
- **Exercício:** Fazer pequenos acréscimos no código para determinar as *articulações* e as *pontes*.

Busca em profundidade

Aplicação 6: Dado um grafo G , determinar as articulações e os blocos de G .

Observação:

- Um filho w de v que satisfaça $back(w) \geq PE(v)$ é chamado de **demarcador de v** .
- **Exercício:** Verificar se cada bloco possui o seu próprio demarcador. (Se isto for verdade, então o número de demarcadores é igual ao número de blocos.)